

شرح مشروع — Distributed Systems Lab

موقع تفاعلي لمحاكاة مفاهيم الأنظمة الموزعة · مبني بـ Next.js + TypeScript + Tailwind

فكرة المشروع

بنيت موقع تفاعلي يشرح مفاهيم الأنظمة الموزعة بشكل عملي بدل النظري. أخذت عشر مفاهيم أساسية وحوّلت كل واحد لمحاكاة حيّة بنشتغل عالمتصفح، المستخدم يلعب فيها بأزرار ومنزقات ويشوف النتيجة لحظياً. كل وحدة بتبّش بعرض المشكلة الحقيقية أول، وبعدين بتخليّ يجرب الحل خطوة بخطوة ويفهم ليش بيشتغل. كل المحاكاة منطق صافي عالمتصفح بدون أي باك-إند، والمشروع منشور على الويب. الفكرة الأساسية إنه كل وحدة بتعالج مشكلة، والمقدمة (Three Horsemen) بتعرض الثلاث مشاكل الكبيرة اللي كل باقي الوحدات حلول إلها.

الوحدات

1 — The Three Horsemen — المقدمة

وحدة المدخل. بتعرض الثلاث مشاكل اللي بيواجهها أي نظام موزّع: **التأخير** (النداء عبر الشبكة مش فوري، وبيتراكم لما النداءات تتسلسل)، **الفشل الجزئي** (جزء يقع والباقي شغال، فتضل مش عارف إذا العملية تمّت)، و**التزامن** (طلبين بيلمسوا نفس البيانات بنفس اللحظة فيصير تضارب، مثل بيع نفس الكرسي مرتين). الهدف إنه الطالب يحسّ بالمشاكل قبل ما يتعلّم حلولها.

2 Load Balancer

المشكلة إنه سيرفر واحد بيصير بطيء تحت الضغط، وإذا وقع بيوقع الموقع كله. الحل موازن أحمال بيوزّع الطلبات على مجموعة سيرفرات. عملت محاكاة بأربع سيرفرات وفيها ست خوارزميات توزيع، وفيك تقتل سيرفر وتشوف الموازن يحوّل الترافيك للباقي فوراً بدون ما يوقف الموقع.

3 Fault Tolerance

المشكلة إنه لما خدمة تبدأ تفشل، الاستمرار بمناداتها بيكدّس المهلات وبينشر الفشل لكل التطبيق (فشل متسلسل). الحل قاطع دارة: بعد عدد أعطال "بيقطع" ويرفض الطلبات فوراً نحو بديل، فبيعطي الخدمة فرصة تتعافى، وبعد مهلة يجربها بطلب واحد بحذر ويرجع يشتغل. معه إعادة محاولة بتأخر متزايد ونبض دوري بيراقب الصحة.

Sharding 4

المشكلة إنه البيانات ما عادت تكفي على جهاز واحد، فبنقسّمها على عدة أجهزة (أقسام). السؤال: أي مفتاح بيروح لأي قسم؟ عرضت أربع استراتيجيات، وأهم نقطة بتظهر بالأرقام: لما تضيف قسم، التجزئة العادية بتعيد خلط كل المفاتيح تقريباً، بينما التجزئة الثابتة بتحرّك جزء صغير بس — ليهيك الأنظمة الكبيرة بتستعملها.

Replication 5

المشكلة إنه نسخة وحدة من البيانات معناها عطل واحد بيضيّعها، وجهاز واحد بيختنق تحت كل القراءات. الحل نخلي نسخ على كذا عقدة. بس النسخ بتتأخر، فلو قرأت من نسخة متأخرة بتطلع بيانات قديمة، ولو وقع الأساسي قبل ما ينسخ آخر كتابات بتضيع. المقايضة: غير متزامن (سريع بس ممكن يضيّع) مقابل متزامن (متين ومتسق بس أبطأ).

Remote Invocation 6

المشكلة إنه مناداة دالة على جهاز ثاني بتبيّن مثل نداء عادي، بس بالحقيقة لازم المعطيات تتسلسل وتعبر الشبكة وترجع. عملت محاكاة بتوزّجي النداء عبر ثماني مراحل، وبتقارن بين RMI و REST و gRPC بالحجم والسرعة. وإذا قطعت الشبكة بالنص بيطلع فشل جزئي: العميل بياخذ خطأ بس ما بيعرف إذا الخادم نفذ الطلب ولا لأ.

Messaging & Pub/Sub 7

المشكلة إنه النداء المباشر يجبر المرسل ينتظر المستقبل، وبينكسر إذا المستقبل واقع. الحل: الناشر يرمي رسالة بوسيط ويكمل شغله، وكل مشترك ياخذ نسخته ويعالجها على وقته. وإذا في رسالة بتفشل دايماً، بتنزل بطابور رسائل ميتة (DLQ) بعد عدد محاولات مشان ما تسدّ الطابور للباقي.

Rate Limiter 8

المشكلة إنه لو إجا للخادم طلبات أكثر من طاقته بينهار ويوقّع الكل. الحل محدّد معدّل يسمح بعدد معيّن من الطلبات ويرفض الزيادة بـ HTTP 429 — يعني نضحي بالقليل لنحمي الكثير. عرضت خوارزميتين: دلو الرموز بيسمح بدفعات قصيرة، والدلو المثقوب بيفرض خرج ثابت تماماً.

Saga 9

المشكلة إنه عملية وحدة (مثل الشراء) بتمتدّ عبر عدة خدمات — طلب، مخزون، دفع، شحن — كل وحدة إلها قاعدتها، فما فينا نلقّهم بمعاملة واحدة ولا نمسك قفل عبر الشبكة. الحل: نشغلهم كسلسلة خطوات محلية، وإذا فشلت خطوة نتراجع عن المكتملة بإجراء تعويضي عكسي (ردّ الدفع، تحرير المخزون) بالترتيب المعكوس. التعويض مش تراجع قاعدة بيانات، هو عكس منطقي حقيقي.

المشكلة إنه لما يموت القائد، مين بياخد محلّه بدون ما يصير قائدين بنفس الوقت يكتبوا بيانات متضاربة؟
الحل تصويت: خمس عُقد بتنتخب قائد واحد بالأغلبية، والقائد بس بيقبل الكتابات وينسخها للباقي. وبما إنّ كل قرار بدّه أغلبية (3 من 5)، مستحيل يصير قائدين بنفس الفترة. فيك تقتل القائد وتشوف العُقد تعمل انتخاب جديد وتختار قائد لحالها.